

De la ES1 a la Eva 2

Cómo llevar el proyecto que ya entregaste hasta la evaluación de esta unidad. Se corrige sobre 30 puntos. No empiezas de cero: tu regla de decisión ya está hecha y no se toca.

Cómo se lee este documento

Usa las mismas cajas que las instrucciones de la ES1, en el mismo orden, en cada bloque:

- **Idea principal** (caja azul): la frase que hay que entender sí o sí.
- **Pasos**: lo que escribes, en orden.
- **Analogía** (caja verde): una comparación con algo del día a día. Es para entender, **no** una definición técnica.
- **Palabras nuevas** (caja café clara): las palabras técnicas de **ese mismo bloque**.

Son 6 bloques. Cada bloque dice en qué número vas y a qué criterio de la rúbrica corresponde.

Esta versión sí usa base de datos. Los datos dejan de guardarse en `datos.json` y pasan a SQLite, que Django ya trae incluido.

Tu regla de decisión no se reescribe y no se copia. Se sigue importando, ahora desde las vistas nuevas. Si terminas con la misma cadena de `if/elif` en dos archivos, hiciste algo mal.

Qué cambia y qué se queda igual

Idea principal: lo que cambia es dónde viven los datos y quién puede tocarlos. Tu función de decisión sigue siendo la misma.

Fase 1 · La base de datos y el modelo

Criterio 2.1.1

Fase 2 · El administrador de Django

Criterio 2.1.2

Fase 3 · Las cuatro operaciones CRUD

Criterio 2.1.3

Fase 4 · Login y roles

Criterio 2.1.4

Fase 5 · Actualizar `ia.md` y `plan.md`

Criterio 2.1.4

Lo que ya tienes y se queda: tu función `decidir()` en `solucion.py`, con sus 4 resultados. Tu `requirements.txt`. Tu `.env` y `.env.example`.

Lo que se va: `datos.json` deja de ser el almacenamiento. La vista de sólo lectura se reemplaza por cuatro vistas.

ANALOGÍA

Hasta ahora anotabas los préstamos en un cuaderno que cualquiera podía leer y rayar. Ahora los anotas en un sistema con fichas ordenadas, con un mesón de atención donde cada persona sólo puede hacer lo que le corresponde: unos anotan, otros sólo miran.

Es una comparación para entender, no una definición técnica.

PALABRAS NUEVAS DE ESTE BLOQUE

Base de datos

Un archivo organizado en tablas donde el programa guarda y busca datos.

SQLite

La base de datos que Django trae lista. Es un solo archivo, no hay que instalar nada.

CRUD

Las cuatro cosas que se le hacen a un dato: crear, leer, editar y eliminar.

Fase 1 · La base de datos y el modelo · Criterio 2.1.1

Idea principal: un **modelo** es la descripción de tu ficha. Cada clave de tu `datos.json` se convierte en un campo.

Paso 1. Abre `settings.py` y confirma que este bloque exista. Django ya lo trae: normalmente no hay que tocarlo.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Paso 2. Mira un registro de tu `datos.json`. Los números van a `IntegerField`, los textos a `CharField`. Si un campo sólo acepta ciertos valores, usa `choices` y la base te los valida sola.

```
# app/models.py
from django.db import models
from django.utils import timezone

class Registro(models.Model):
    ESTADO_CHOICES = [("al dia", "Al día"), ("moroso", "Moroso")]

    nombre      = models.CharField(max_length=100)
    cantidad    = models.IntegerField()
    estado      = models.CharField(max_length=20, choices=ESTADO_CHOICES)
    resultado   = models.CharField(max_length=200)
    fecha       = models.DateTimeField(default=timezone.now)

    # Borrado logico: no se pierde nada, solo se oculta
    eliminado    = models.BooleanField(default=False)
    fecha_eliminacion = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-fecha"]

    def __str__(self):
        return f"{self.nombre} - {self.resultado}"

    def soft_delete(self):
        self.eliminado = True
        self.fecha_eliminacion = timezone.now()
        self.save() # sin esto no guarda nada
```

Paso 3. Registra la app en `INSTALLED_APPS` y crea las tablas:

```
python manage.py makemigrations
python manage.py migrate
```

Paso 4. Comprueba que quedó bien. Este comando no debe detectar nada:

```
python manage.py makemigrations --check --dry-run
# debe responder: No changes detected
```

Si detecta cambios, significa que modificaste el modelo y te falta crear la migración.

ANALOGÍA

El modelo es el formulario en blanco que define qué casillas tiene una ficha. La migración es la orden de imprimirlo: hasta que no la ejecutas, el archivador no tiene dónde guardar nada.

PALABRAS NUEVAS DE ESTE BLOQUE

Modelo

Una clase de Python que describe qué campos tiene un dato guardado.

Campo

Cada casilla de la ficha: nombre, cantidad, estado.

Migración

El archivo que traduce tu modelo a tablas reales en la base.

Borrado lógico

Marcar una ficha como eliminada en vez de borrarla. Queda el registro.

Fase 2 · El administrador de Django · Criterio 2.1.2

Idea principal: registrar el modelo es el mínimo. Lo que se evalúa es que el administrador sirva para trabajar.

Paso 1. Configura las columnas, el filtro y el buscador:

```
# app/admin.py
from django.contrib import admin
from .models import Registro

@admin.register(Registro)
class RegistroAdmin(admin.ModelAdmin):
    list_display = ("nombre", "estado", "cantidad", "resultado", "fecha")
    list_filter = ("estado", "eliminado")
    search_fields = ("nombre",)
    readonly_fields = ("fecha_eliminacion",)
```

Paso 2. Crea tu usuario administrador:

```
python manage.py createsuperuser
```

Paso 3. Entra a `/admin/` y revisa tres cosas: que tu modelo aparezca en la portada, que la lista muestre columnas y no `Registro object (1)`, y que el filtro lateral y el buscador funcionen.

ANALOGÍA

El administrador es la trastienda del local. El público entra por la puerta principal y ve lo que preparaste; tú entras por atrás y ves todo el inventario, incluso lo que está fuera de exhibición.

PALABRAS NUEVAS DE ESTE BLOQUE

Superusuario

La cuenta con permiso para entrar al administrador y hacer todo.

`list_display`

Las columnas que se muestran en la lista del administrador.

Fase 3 · Las cuatro operaciones CRUD · Criterio 2.1.3

Idea principal: cuatro vistas, una por operación. La línea que importa `decidir` es donde reutilizas lo que ya hiciste.

```
# app/views.py
from django.shortcuts import render, redirect, get_object_or_404
from solucion import decidir          # tu regla de la ES1, intacta
from .models import Registro

def lista(request):                    # READ
    registros = Registro.objects.filter(eliminado=False)
    return render(request, "lista.html", {"registros": registros})

def crear(request):                   # CREATE
    error = None
    if request.method == "POST":
        nombre = request.POST.get("nombre", "").strip()
        try:
            cantidad = int(request.POST.get("cantidad", ""))
        except ValueError:
            error = "La cantidad debe ser un numero entero."
        else:
            estado = request.POST.get("estado", "").strip().lower()
            resultado = decidir(cantidad, estado)
            Registro.objects.create(nombre=nombre, cantidad=cantidad,
                                    estado=estado, resultado=resultado)
            return redirect("lista")
    return render(request, "form.html", {"accion": "Crear", "error": error})

def editar(request, pk):              # UPDATE
    reg = get_object_or_404(Registro, pk=pk, eliminado=False)
    if request.method == "POST":
        reg.nombre = request.POST.get("nombre", "").strip()
        reg.cantidad = int(request.POST.get("cantidad"))
        reg.estado = request.POST.get("estado", "").strip().lower()
        reg.resultado = decidir(reg.cantidad, reg.estado) # recalcular
        reg.save()
        return redirect("lista")
    return render(request, "form.html", {"accion": "Editar", "registro": reg})

def eliminar(request, pk):            # DELETE logico
    reg = get_object_or_404(Registro, pk=pk, eliminado=False)
    if request.method == "POST":
        reg.soft_delete()
        return redirect("lista")
    return render(request, "confirmar.html", {"registro": reg})
```

Y las rutas en `urls.py` :

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("registros/", views.lista, name="lista"),
    path("registros/crear/", views.crear, name="crear"),
    path("registros/<int:pk>/editar/", views.editar, name="editar"),
    path("registros/<int:pk>/eliminar/", views.eliminar, name="eliminar"),
]
```

Dos errores que se repiten. En el update, vuelve a llamar a `decidir()` : si cambias la cantidad y dejas el `resultado` viejo, la ficha queda mintiendo. Y valida antes de escribir: un `int()` sin `try/except` devuelve error 500 apenas alguien escriba letras.

Para no entregar una tabla vacía, pasa tus datos viejos del JSON a la base:

```
# cargar_datos.py, se corre con:
# python manage.py shell < cargar_datos.py
import json
from app.models import Registro

for r in json.load(open("datos.json", encoding="utf-8")):
    Registro.objects.create(nombre=r["nombre"], cantidad=r["cantidad"],
                             estado=r["estado"], resultado=r["resultado"])
```

ANALOGÍA

Es el mesón de una biblioteca. Crear es inscribir un libro nuevo, leer es mirar el catálogo, editar es corregir una ficha mal escrita, y eliminar es sacar el libro de circulación sin quemarlo: la ficha queda en el archivo histórico.

PALABRAS NUEVAS DE ESTE BLOQUE

Vista

La función que responde cuando alguien entra a una dirección.

ORM

La forma de hablarle a la base con Python, sin escribir SQL. Es el `.objects.create()`.

get_object_or_404

Busca una ficha y, si no existe, muestra un error 404 en vez de romperse.

Fase 4 · Login y roles · Criterio 2.1.4

Idea principal: Django ya trae el sistema de usuarios. No escribas el tuyo y **nunca** guardes contraseñas en un modelo propio.

Paso 1. Entrar y salir:

```
from django.contrib.auth import authenticate, login, logout

def vista_login(request):
    if request.method == "POST":
        user = authenticate(request,
                             username=request.POST.get("username", "").strip(),
                             password=request.POST.get("password", ""))
        if user:
            login(request, user)
            return redirect("lista")
        messages.error(request, "Usuario o contraseña incorrectos.")
    return render(request, "login.html")

def vista_logout(request):
    logout(request)
    return redirect("login")
```

Paso 2. Los tres roles se arman con **grupos**, no con un campo tuyo:

```
# crear_usuarios.py
import os
from django.contrib.auth.models import User, Group

for nombre in ("admin", "normal", "viewer"):
    Group.objects.get_or_create(name=nombre)

# La contraseña sale del .env, NO escrita aquí
u = User.objects.create_user("lector", password=os.environ["PASS_LECTOR"])
u.groups.add(Group.objects.get(name="viewer"))
```

Paso 3. El decorador que protege cada vista:

```

from functools import wraps
from django.contrib.auth.decorators import login_required

def tiene_rol(user, *roles):
    return user.groups.filter(name__in=roles).exists() or user.is_superuser

def requiere_rol(*roles):
    def decorador(view_func):
        @wraps(view_func)
        @login_required(login_url="login")
        def wrapper(request, *args, **kwargs):
            if tiene_rol(request.user, *roles):
                return view_func(request, *args, **kwargs)
            messages.error(request, "No tienes permiso para esta accion.")
            return redirect("lista")
        return wrapper
    return decorador

```

Paso 4. Lo pones encima de cada vista, según quién puede qué:

```

@login_required(login_url="login")
def lista(request): ...          # todos los que entraron

@requiere_rol("admin", "normal")
def crear(request): ...         # admin y normal

@requiere_rol("admin")
def editar(request, pk): ...    # solo admin

@requiere_rol("admin")
def eliminar(request, pk): ...  # solo admin

```

Esto es lo que más se evalúa. Esconder un botón con `{% if es_admin %}` **no es seguridad**: cualquiera puede escribir la dirección a mano y saltarse el botón que no ve. La restricción va en el servidor, en el decorador. El `{% if %}` está bien, pero va *además*, nunca en lugar del decorador.

Paso 5. Revisa que **todos** tus formularios POST tengan `{% csrf_token %}` dentro del `<form>`. Sin eso Django rechaza el envío con error 403.

Paso 6. Pruébalo de verdad. Entra con el usuario de menor rango y escribe a mano en el navegador `/registros/1/editar/`. Te tiene que rechazar. Si te deja entrar, tu seguridad está sólo en la plantilla.

ANALOGÍA

Es la diferencia entre sacar el botón del ascensor que va al piso 10 y no darle la llave a nadie que no corresponda. Si sólo sacas el botón, el que conoce el edificio igual sube por la escalera.

PALABRAS NUEVAS DE ESTE BLOQUE

Autenticación

Comprobar quién eres: usuario y contraseña.

Autorización

Comprobar qué te dejan hacer una vez que entraste. Son cosas distintas.

Grupo

La forma de Django de juntar usuarios bajo un mismo rol.

Decorador

Una función que envuelve a otra para revisar algo antes de dejarla correr.

CSRF

Protección que impide que otra página envíe formularios en tu nombre.

Fase 5 · Actualizar ia.md y plan.md · Criterio 2.1.4

Idea principal: es el paso que más gente se salta, y vale un tercio del criterio más pesado. Tus dos documentos de la ES1 describen un sistema que **ya no es** el que estás entregando.

plan.md dice hoy: «sin base de datos», «login» en Won't

Corregir

ia.md dice hoy: la consulta sobre la cadena if/elif

Corregir

En plan.md, actualiza el alcance real: SQLite, CRUD, roles y borrado lógico. Mueve el login de la columna Won't a la de Must.

En ia.md escribe tres cosas, cortas y concretas: **qué herramienta** usaste, **qué le preguntaste** textualmente sobre el modelo, las migraciones o los roles, y **qué corregiste** de lo que te respondió.

Lo tercero es lo que vale. Si la IA te propuso guardar contraseñas en tu propio modelo, o usar PostgreSQL, o resolver los permisos sólo con `{% if %}` en la plantilla, eso es exactamente lo que hay que escribir.

ANALOGÍA

Es la memoria de cálculo de un plano. La casa puede estar bien construida, pero si entregas la memoria del proyecto anterior, nadie puede revisar que lo que hiciste corresponde a lo que se pidió.

PALABRAS NUEVAS DE ESTE BLOQUE

Alcance

El límite de lo que tu programa hace y de lo que no hace.

MoSCoW

Forma de ordenar funciones en cuatro grupos según qué tan necesarias son.

Antes de entregar

Los cuatro primeros puntos hundieron entregas de la ES1 que por lo demás estaban bien resueltas.

- ❑ Tu `.gitignore` está en **UTF-8, no en UTF-16**. Si lo creaste en PowerShell con `>`, git no puede leerlo y no ignora nada. En VS Code la codificación se ve y se cambia abajo a la derecha.
- ❑ El `.env` no está en el repositorio y la `SECRET_KEY` no está escrita dentro de `settings.py`. Verifícalo con `git ls-files | grep env`. Si alguna vez la subiste, **regénerala**: borrarla después no la saca del historial.
- ❑ El `.env.example` tiene contenido: los nombres de las variables, sin los valores. Un archivo vacío no le dice nada a nadie.
- ❑ No subiste `venv/` ni `db.sqlite3` con usuarios reales. Si además el script de usuarios trae las contraseñas escritas, cualquiera que clone entra como administrador.
- ❑ El proyecto levanta desde un clon limpio: otra carpeta, instalar `requirements.txt`, copiar `.env.example` a `.env` y correr `migrate`.
- ❑ `makemigrations --check` no detecta cambios.
- ❑ Probaste los permisos escribiendo la dirección a mano, no sólo mirando si el botón aparece.
- ❑ Borraste lo que ya no se usa: proyectos duplicados, apps vacías, plantillas sin ruta, dependencias que ya no importas.
- ❑ `ia.md` y `plan.md` hablan de **esta** entrega, no de la anterior.